Traffic Scene Understanding on Embedded Systems using Computer Vision and Deep Learning Methods

Mikel Dudi, Department of Electrical & Computer Engineering, University of Peloponnese, Patras, Greece, ece21110@go.uop.gr

***Abstract*— This project presents a hybrid system for real-time road sign and zebra crossing detection on embedded hardware. A fine-tuned YOLOv8 model is employed for traffic sign detection, focusing on small objects such as stop signs, speed limits, and traffic lights. In parallel, a lightweight computer vision pipeline identifies zebra crossings by extracting bright white regions, grouping them into stripe patterns, and highlighting confirmed crossings. The system is deployed on a Raspberry Pi, demonstrating efficient, real-time performance. By combining machine learning with classical vision methods, this project contributes a practical pattern recognition solution for road safety in dynamic environments.**

Keywords—YOLO, deep learning, computer vision

I. INTRODUCTION

Deep learning is proving its worth every year in many vision based applications, including vital ones like traffic monitoring and autonomous driving. In this field, semantic road identification and precise traffic sign recognition are becoming essential elements for enhancing computer vision safety [1]. The development of advanced driving assistance systems (ADAS) and self-driving cars depends on efficient object detection. Although many algorithms have been developed for reliable object detection using machine learning techniques, the challenges to traffic scene understanding are currently being studied. Detection of traffic scenes by identifying features such as traffic signs, pedestrians, and zebra crossings ensures road safety. Traffic signs provide necessary regulating and warning information to control drivers' actions, and pedestrians are the most vulnerable of all road users whose safety relies on time-critical identification and response. Zebra crossings offer certain safe zones through which traffic has to cede way, and precise identification of these zones becomes paramount in accident prevention. Regular identification of these features not only provides the foundation for intelligent transportation systems and driver-assistance systems but also prevents accidents, enhances pedestrians' safety, and fosters safer and more effective urban transportation.

Prior research has investigated traffic scene understanding through both traditional computer vision approaches and lightweight deep learning frameworks. For pedestrian detection, [2] compared techniques such as background subtraction, Haar cascades, and HOG with SVM classifiers, showing that while HOG + SVM achieves acceptable performance in controlled environments, its robustness diminishes under scale variation, occlusion, and complex backgrounds. In contrast, YOLO-based detectors exhibit superior generalization by leveraging end-to-end feature learning and multi-scale prediction, thereby attaining higher recall and localization accuracy. Similarly, [3] conducted a comparative analysis of MobileNet-SSD and YOLO, concluding that although MobileNet-SSD offers reduced computational cost and faster inference on embedded systems, YOLO consistently outperforms it in terms of mean average precision and the reliable detection of small objects such as traffic signs. By adopting YOLO architectures for both pedestrian and traffic sign detection, the present work seeks to capitalize on these advantages to advance the accuracy and reliability of traffic scene understanding systems, ultimately contributing to safer and more intelligent transportation environments.

This study presents a hybrid approach to real-time road sign and zebra crossing detection, integrating machine learning with classical computer vision techniques. The traffic sign detection module employs a fine-tuned YOLOv8 model, optimized for small object detection, to identify stop signs, speed limits, traffic lights, and crosswalk signs. In parallel, zebra crossings are detected without machine learning, using a computer vision pipeline designed for efficiency, similar to methods described in [4]. The system analyzes the lower region of each frame, extracts bright white areas through HSV thresholding, applies morphological operations to ensure stripe continuity, and localizes candidate crossings via contour and geometric filtering. Groups of aligned stripes meeting size and spacing criteria are classified as zebra crossings and displayed with bounding boxes and textual indicators.

The combined system is implemented on a Raspberry Pi4, demonstrating real-time performance and robustness under varying conditions. By addressing the challenges of recognizing structured road patterns in dynamic environments, this work contributes a lightweight and practical solution to embedded pattern recognition for road safety applications. In this project, we propose a materially limited, yet efficient approach to tackle with real-time problems utilizing cheap hardware with software-based optimizations.

II. CHALLENGES

Traffic Sign Recognition is about understanding vision based real life scenarios in an artificially controlled environment. While limitations help engineers in Autonomous Vehicle Driving by having an extensive knowledge on traffic situations when traffic rules are not broken, real life scenarios still differ in many ways.

- One of the most discussed problem is how to detect and to compensate for the change in ambient lighting conditions,

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

including weather changes, daylight, and vehicle turns. Cameras are optimized for human view (in)abilities, so they change displayed color representations according to lighting angle and brightness. For example, cameras add the least important color (usually blue) to capture the sense of shade, fog, rainy, or cloudy weather, or to compensate greyish patterns in images. If cameras are set against incoming lighting, i.e. one drives against the Sun, cameras compensate high luminance values to white by turning all colors into grey. One must add there is almost no project on night time condition TSR.

- Occlusions of traffic signs due to the presence of objects such as trees, buildings, vehicles, pedestrians, or other signs are also important factor to be considered. If part of the object is covered then successful detection and classification require recognition tasks to relax some preconditions that also increase the number of non traffic sign objects to be recognized as road signs. A general approach is to calculate and record a large number of features on image objects, however, because of the computational complexity real-time criteria can be met only at a very low speed or frame rate.
- Different in-plane and out-of-plane rotations, and the viewing angle can cause problems for recognition as both shape based recognizer, and classification methods are not robust enough for n-dimension non-affine transformations. For example, squared traffic signs are rather look like trapezes from the driver point of view, and even circles turn to be ovals according to cameras non-linear optical transformations.
- Aging and vandalism also affect the image perception. Deterioration or intentional deformations (e.g. corrosion, paintings) result in either a loss or a misinterpretation of information. These problems can not be resolved using camera systems only.

III. METHODOLOGY

A real-time autonomous driving system must face all formentioned problems while they solve traffic sign detection and recognition, pedestrian and zebra crossing detection.

A. Traffic Sign Detection Model

The detection of road signs was implemented using a fine-tuned version of the Ultralytics YOLOv8n model. The dataset used for training consisted of approximately 900 images containing stop signs, speed limit signs, crosswalk signs, and traffic lights captured under diverse conditions, including varying angles, lighting, and backgrounds. The dataset was divided into 80% training and 20% validation subsets. Training was performed for 50 epochs with an input image resolution of 320 × 320 pixels.

YOLO is a state-of-the-art technology developed for object detection based on DL, with emphasis on real-time and high-accuracy applications [1]. In YOLO, object detection is treated as a regression problem where candidate images and their categories and confidence indices are directly generated by regression. The detection result is finally determined by setting a threshold of the confidence rate and the non-maximum suppression technique.

The primary advantage of YOLO lies in its capability for real-time image processing, making it well-suited for applications like autonomous vehicles and Advanced Driver Assistance Systems (ADAS). Moreover, YOLO achieves state-of-the-art accuracy with limited training data, surpassing other methods. Additionally, its ease of implementation and user-friendliness contribute to its popularity in the field of computer vision (CV).

After training, the model was converted into the **NCNN format** to optimize inference performance on resource-constrained hardware, specifically the Raspberry Pi 4.

The detection pipeline follows a standard sequence of steps. First, input frames are captured from the camera and resized to the model's resolution. The preprocessed frames are then passed through the YOLOv8 network, which outputs bounding boxes and confidence scores for candidate detections. A non-maximum suppression (NMS) step removes redundant overlapping predictions. Finally, the resulting bounding boxes are visualized on the frame with labels corresponding to the detected traffic sign type.

This approach enables real-time detection of multiple types of road signs while maintaining computational efficiency suitable for embedded deployment.

B. Pedestrian Detection

Pedestrian detection was implemented using the pretrained Ultralytics YOLOv8n model. Unlike the traffic sign detector, this network was not fine-tuned, as the **COCO dataset** already includes a large number of annotated pedestrian instances suitable for the project's requirements. The YOLOv8n variant was selected due to its lightweight architecture, offering a balance between detection accuracy and inference speed, which is critical for deployment on resource-constrained hardware.

Similar to the traffic sign detector, the model was exported to the **NCNN format** to enable efficient inference on the Raspberry Pi 4. The detection pipeline follows the same sequence of steps: input frames are resized and normalized, passed through the YOLOv8n network, and post-processed using non-maximum suppression to eliminate duplicate detections. The final bounding boxes corresponding to the "person" class are then visualized on the frame.

This module provides reliable pedestrian localization in real time, complementing the road sign and zebra crossing detection systems to form a more complete traffic scene understanding.

C. Zebra Crossing Detection

Unlike traffic sign and pedestrian detection, which rely on deep learning, zebra crossings were detected using a traditional computer vision pipeline designed for efficiency and robustness on embedded hardware. The method

leverages the geometric and brightness characteristics of zebra crossings rather than learning from data, making it lightweight and well-suited for real-time inference.

The pipeline begins by selecting a **Region of Interest (ROI)** corresponding to the lower portion of each frame, where zebra crossings are most likely to appear. The ROI is then converted into the **HSV color space**, and a thresholding operation isolates bright white regions corresponding to potential crossing stripes. A **morphological closing operation** is applied to reduce noise and ensure continuity of the extracted regions.

Next, **contour detection** is performed to localize candidate stripes. These contours are filtered by geometric criteria: only wide horizontal regions are retained, with aspect ratios between 2 and 15 and a minimum width greater than 40 pixels. The remaining stripes are grouped vertically, and groups containing at least three aligned stripes are classified as zebra crossings. For visualization, detected crossings are highlighted with bounding boxes and a textual indicator ("ZEBRA CROSSING: YES/NO") overlaid on the frame.

This classical approach complements the deep learning models by detecting structured pedestrian crossings without requiring additional training data, thereby ensuring low computational cost and real-time performance on the Raspberry Pi 4.

IV. PROPOSED SYSTEM ATTEMPT

Our system architecture block diagram is in Figure 3. For the system, we used a Raspberry Pi4, equipped with a 720p Webcam for testing on the car's dashboard. For the pedestrian auto brake input system, a ROI is designed on the program's monitor, ensuring that the car stops only when pedestrians are detected on the road, the system warns the car and driver to brake.

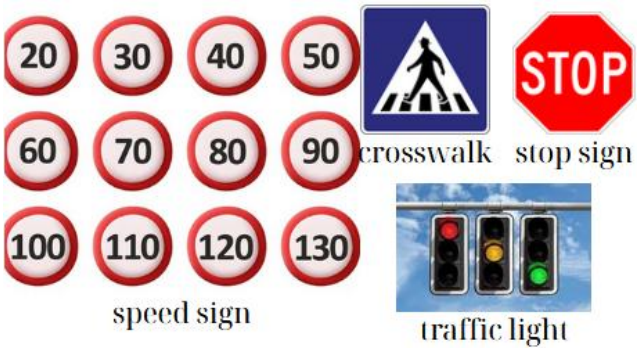

Fig. 1. 4 Classes used for the evaluation of the Road Sign Detection dataset.

TABLE I. HARDWARE CONFIGURATION

Environment	Configuration Information
Operating System	Pi OS Desktop
RAM	8GB
CPU	Quad-core ARM Cortex-A72 1.5GHz
Storage	128GB Micro SD

A. Evaluation Metrics

- To evaluate the performance of our fine-tuned Traffic Sign Detection model, we employ multiple standard object detection metrics. A **confusion matrix** is presented to illustrate the distribution of true positives, false positives, false negatives, and true negatives across different traffic sign classes. In addition, we compute **Precision, Recall, and F1-scores**, which provide a balanced assessment of the model's detection capabilities. Precision reflects the reliability of the predicted detections, Recall indicates the ability of the model to capture all relevant traffic signs, and the F1-score serves as a harmonic mean that balances both. To further analyze model behavior, **Precision, Recall, and F1 curves** are plotted, offering insights into performance under varying confidence thresholds. These curves show false positive behavior, false negative behavior, and the best balance point for the proposed system accordingly. Together, these metrics provide a comprehensive understanding of the model's accuracy and robustness under diverse conditions. The equations to calculate these 3 are shown below:

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F1 = 2 * \frac{Precision * Recall}{Precision + Recall}$$

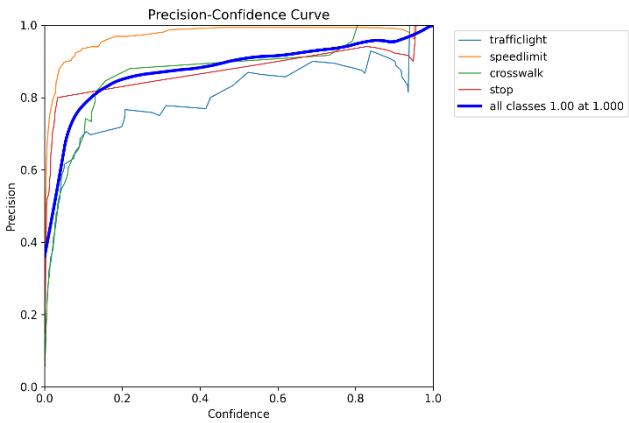

Fig. 2. Precision – Confidence Curve after model training.

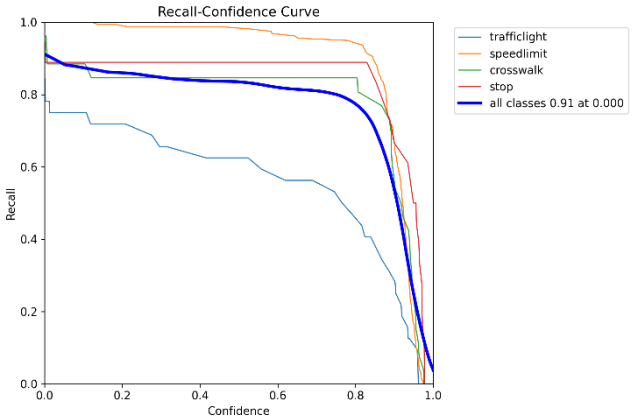

Fig. 3. Recall – Confidence Curve after model training.

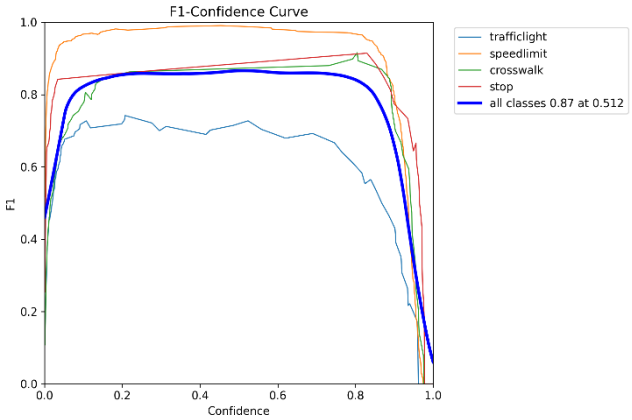

Fig. 4. F1 – Confidence Curve after model training.

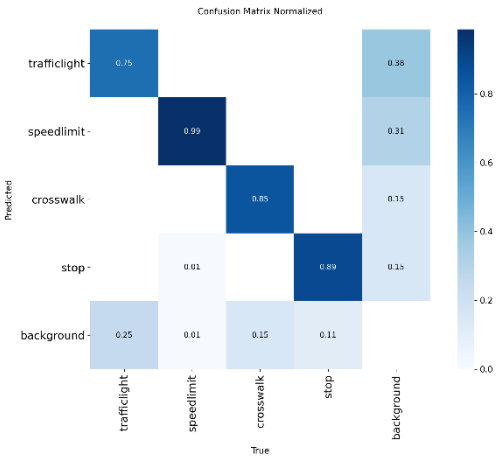

Fig. 5. Confusion matrix after model training.

B. *Experimental Results*

During evaluation on the Raspberry Pi 4, the individual detection modules demonstrated varying levels of real-time capability. The fine-tuned YOLOv8 traffic sign detector and the pretrained YOLOv8n pedestrian detector achieved frame rates between **5.5–8 FPS** when running on live webcam input, while the zebra crossing detection pipeline based on HSV thresholding and morphological operations was significantly lighter, maintaining **15–25 FPS** in the same conditions. Each module preserved high detection accuracy under diverse environments. When the three methods were integrated and executed concurrently on prerecorded **.mp4 video input**, the overall system frame rate decreased to **4.5–6 FPS**. Despite this reduction in speed, detection accuracy across all modules remained stable, demonstrating that the hybrid pipeline can still provide comprehensive traffic scene understanding on embedded hardware while preserving practical real-time feasibility. This trade-off highlights the computational cost of running multiple detection methods simultaneously, but confirms the system's potential for real-world deployment. Furthermore, the average inference time per frame was measured at **<0.2 seconds** for the YOLO-based modules, corresponding to a throughput of approximately **5–8 FPS** on the Raspberry Pi 4 based on the mathematical calculation below:

$$FPS = \frac{1}{Inference\ Time(s)}$$

Fig. 6. Samples of tested images.

Fig. 6. Samples of tested images on lower lighting conditions.

Fig. 7. The HSV Zebra Crossing Detector tested in .mp4 video.

V. Conclusion and Future Work

For our project, we implemented a hybrid traffic scene interpretation system combining deep learning with conventional computer vision. With a traffic sign recognition with a YOLOv8 model finetuned with our data, a pedestrian recognizer with a pretrained YOLOv8n model, and an inexpensive pipeline with HSV for zebra crossings, we were able to implement all three systems on a Raspberry Pi 4 in real-time. In results, we determined that consistent detection of crucial traffic objects is achievable with frame rates from 4.5–8 FPS depending on configuration. This proves even low-power systems can enable practical real-time detection with applications to driver assistance or low-budget autonomous systems.

But still, the system is not perfect. It is also too intensive to support running multiple YOLO models simultaneously with a single Raspberry Pi since it brings down the frame rate significantly. It also fares badly with difficult conditions where much is occluded or where lighting is low or blurred. In addition to that, the traffic sign detection module only identifies four categories but does not discriminate signs down to detail, including different speed limits or traffic light conditions.

In the future, a few things that can be attempted to increase this work include increasing speed with optimization strategies such as pruning or quantization or with accelerators such as Google Coral TPU or NVIDIA Jetson. Increasing the size of the data to train the models with more traffic sign classes would increase accuracy and robustness. Addition of tracking over frames would also reduce false detection. Advanced approaches such as semantic segmentation would be able to scale the system beyond objects to a fuller road scene understanding.

References

[1] M. Flores-Calero, C. A. Astudillo, D. Guevara, J. Maza, B. S. Lita, B. Defaz, J. S. Ante, D. Zabala-Blanco, and J. M. Armingol Moreno, “Traffic Sign Detection and Recognition Using YOLO Object Detection Algorithm: A Systematic Review,” *Mathematics*, vol. 12, no. 2, article 297, Jan. 2024. DOI: 10.3390/math12020297.

[2] S. J. Dutta, “Pedestrian Detection With 4 Different Computer Vision Techniques,” *Medium*, Mar. 28, 2024. [Online]. Available: https://medium.com/@sanjay_dutta/pedestrian-detection-with-4-different-computer-vision-techniques-da046273fee7

[3] S. N. Sabina, A. M. P., and H. P. V., “Object Detection using YOLO and MobileNet SSD: A Comparative Study,” *International Journal of Engineering Research & Technology (IJERT)*, vol. 11, no. 06, June 2022.

[4] D. Ahmetović, V. Grišostas, T. T. Nguyen, and J. Erfović, “Zebra Crossing Spotter: Automatic Population of Spatial Road Databases for Urban Crosswalks,” *PLOS ONE*, vol. 10, no. 5, May 2015. [Online]. Available: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC4725710/